Thunderbolt: Fast Asynchronous Off-Chain Bitcoin Transfers

Anonymous Author(s)

Abstract

We present Thunderbolt, an off-chain protocol that transfers Bitcoin UTXO ownership in ≈**1 s**, requires **no channel graph, no routing, and no liquidity rebalancing**, and lets the **recipient be offline** at the time of transfer. A single UTXO is locked once on-chain under a fixed public key; ownership then passes through an unbounded sequence of holders—each transfer is a purely off-chain, asynchronous operation whose on-chain cost is zero. The chain sees exactly two transactions regardless of how many transfers occur.

The cryptographic core is a pair of Sigma-protocol NIZK proofs—the *Invoice Consistency Proof* and the *Transfer Consistency Proof*—that jointly enforce a "one add, one subtract" invariant on an additively shared key. Both instantiate Camenisch–Shoup verifiable encryption of discrete logarithms without trusted setup or circuit-level tooling. When the committee is a threshold group, Simple VESS (Verifiable Encrypted Secret Sharing) distributes the committee share under a standard (t, n)-honesty assumption—the only trust Thunderbolt introduces beyond the base chain, in exchange for eliminating routing and liquidity constraints entirely.

On a commodity laptop, a complete off-chain transfer finishes in **1.01 s** with a combined proof size of **3.8 KB**. Compared with Lightning, Thunderbolt delivers **2.5–13× lower latency** with deterministic, route-independent timing, eliminates directional liquidity constraints, and removes simultaneous-online requirements. Against SNARK backends on the same relations, our proofs are **100–1000× faster**, as Paillier arithmetic over $\mathbb{Z}_{N^2}$ is worst-case for circuit-based systems.

CCS Concepts

• **Security and privacy** → **Cryptography**; **Distributed systems security**.

Keywords

Bitcoin, Off-Chain Transfers, Verifiable Encryption, Paillier Cryptosystem, Zero-Knowledge Proofs, Payment Channels

ACM Reference Format:

Anonymous Author(s). 2026. Thunderbolt: Fast Asynchronous Off-Chain Bitcoin Transfers. In *Proceedings of Proceedings of the 2026 ACM Conference on Computer and Communications Security (CCS '26).* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/XXXXXXX.XXXXXXX

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CCS '26, The Hague, The Netherlands

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/10.1145/XXXXXXX.XXXXXXX

1 Introduction

Bitcoin [30] intentionally constrains throughput and expressiveness at the base layer: blocks are small, scripts are limited, and every state change pays the cost of global verification. This conservatism is a feature, not a bug—it is what gives Bitcoin its credibility as a settlement layer. The price, however, is that Bitcoin is not directly usable as a fast, low-cost medium for everyday value transfer. For nearly a decade, the dominant answer to this gap has been the *payment channel*: two (or more) parties lock funds in an on-chain UTXO, exchange signed off-chain updates, and fall back to the chain only to open or close the channel. The Lightning Network [35] scales this idea to a routed graph of channels and is today the most widely deployed Bitcoin layer-2.

Lightning's design space, however, bakes in a strong structural assumption: *every payment must traverse a path of pre-funded, bidirectionally-online channels.* This has three well-known consequences that are increasingly at odds with how users actually want to move Bitcoin. First, **connectivity is fragile**: a payer can pay a payee only if a route of sufficient capacity exists at that moment, and route-finding is an open research problem as the graph grows and churns. Second, **liquidity is illiquid**: channel capacity is directional, and imbalanced channels must be rebalanced via on-chain or loop-out operations, which reintroduces the very on-chain cost Lightning tries to avoid. Third, and most fundamentally, **both endpoints must be online and coordinating at the moment of payment**: Lightning is an inherently synchronous, multi-hop HTLC protocol. For a sender paying a cold wallet, a payroll batch crediting thousands of recipients, or a recipient whose device wakes up only once a day, these assumptions are a poor match.

A different question. Instead of asking "how do we route a payment across a graph of pre-funded channels," we ask: *can the ownership of a single Bitcoin UTXO be transferred off-chain, asynchronously, from an arbitrary current holder to an arbitrary next recipient, without ever touching the chain and without either party being online at the same time as the other?* If the answer is yes, then the on-chain footprint of any sequence of transfers—no matter how long—remains one lock, one unlock. The chain sees only the endpoints; everything in between is off-chain state.

Thunderbolt. This paper presents Thunderbolt, an off-chain protocol that answers the above question affirmatively. In Thunderbolt, a Bitcoin UTXO is locked once, under a fixed public key $P = (x_0 + y_0)\,G$ that is additively shared between a *holder* and a *Committee*. Ownership of the UTXO is then transferred through a sequence of "one add, one subtract" updates: at each step s, a fresh refresh value r_s is added to the holder share and subtracted from the committee share, so that $x_{s+1} + y_{s+1} = x_s + y_s$ and the on-chain key P is preserved. The holder share itself lives inside a Paillier ciphertext that travels from one recipient to the next, while the committee share is maintained (or resharing) by a committee that never sees the holder's secret. Crucially, a transfer from Alice to Bob requires no interaction between Alice and Bob beyond

Bob posting a one-time *invoice*; neither party needs to be online simultaneously, and no route through a channel graph is needed. The invoice-and-consume interaction is *recursive*: Bob, once he has consumed Alice's transfer, can immediately serve as the next sender to Carol without any on-chain activity.

Core cryptographic content. The correctness of a single transfer step rests on two non-interactive zero-knowledge (NIZK) proofs: the *Invoice Consistency Proof* and the *Transfer Consistency Proof*.

- The *Invoice Consistency Proof*, produced by the recipient Bob when he issues an invoice, certifies that a single integer r_s is simultaneously (i) the discrete log of the public delta point $D_s = r_s H$, (ii) the Paillier plaintext of the ciphertext ct_B to Bob, and (iii) the Paillier plaintext of the ciphertext ct_C to the Committee. A malicious Bob who deviates from this—say, by committing different refresh values to different parties—is caught by verification.
- The *Transfer Consistency Proof*, produced by the sender Alice when she consumes Bob's invoice, certifies that the integer Alice injects into Bob's next holder ciphertext T_{s+1} is the *actual* current holder secret x_s opening the public anchor $X_s = x_s H$. Alice cannot substitute a fabricated x'_s without the verifier noticing.

Both proofs are direct Sigma-protocol instantiations of *verifiable encryption of discrete logarithms* in the Camenisch–Shoup style [10], compiled to NIZK via Fiat–Shamir. The Invoice Consistency Proof additionally uses a single shared Sigma response z across one elliptic-curve relation and two Paillier relations, which is what ties the three invoice objects to the same r_s. The third-party-side update may alternatively be realized by a threshold group using *Simple VESS* [40], a verifiable encryption of Shamir secret sharing designed as a building block for distributed key generation and resharing.

Why the off-the-shelf primitives matter. The Thunderbolt construction deliberately avoids introducing a new, bespoke cryptographic assumption. Both ingredients—Camenisch–Shoup verifiable encryption and Simple VESS—are published, peer-reviewed, and have been in use for more than two decades in threshold signing, escrow, and distributed key generation. What Thunderbolt contributes is not a new primitive but a new *usage scenario*: we show that the combination of these two existing primitives, specialized to the "one add, one subtract" invariant, is exactly what is needed to implement asynchronous recursive UTXO ownership transfer under a fixed on-chain lock key. As a consequence, Thunderbolt inherits its soundness, zero-knowledge, and extraction arguments directly from the established literature.

Why not a general-purpose SNARK? An obvious alternative is to express the Invoice and Transfer relations as arithmetic circuits and feed them into a general-purpose SNARK backend (e.g., a zkVM such as SP1 [42]). Such a construction would work in principle, but Paillier operations in $\mathbb{Z}_{N^2}$ for a 3072-bit N and secp256k1 scalar multiplication are worst-case workloads for circuit backends, inflating proving time by orders of magnitude compared to our direct Sigma-protocol approach. We make this comparison concrete in Section 6.

Contributions. The main contributions of this paper are:

(1) We introduce Thunderbolt, an off-chain protocol that enables asynchronous, recursive transfer of Bitcoin UTXO ownership between uncoordinated users under a fixed on-chain lock key.
(2) We give two compact Sigma-protocol-based NIZK proofs—the Invoice Consistency Proof and the Transfer Consistency Proof—that jointly certify a single correct transfer step. Both are direct instantiations of Camenisch–Shoup verifiable encryption of discrete logarithms, with the Invoice proof using a single shared Sigma response to bind three invoice objects to the same refresh value r_s.
(3) We formalize Thunderbolt as a payment channel and contrast it with the Lightning Network along the dimensions of connectivity, routing, channel liquidity, and online-ness assumptions.
(4) We compare our Sigma-protocol construction against general-purpose SNARK backends on the same statements and show that it yields proving and verification costs orders of magnitude smaller than a circuit-based realization.

Roadmap. Section 2 recalls background primitives. Section 3 motivates the VEDL + VESS composition. Section 4 presents the Invoice and Transfer Consistency Proofs with full security arguments. Section 5 formalizes Thunderbolt as a payment channel. Section 6 benchmarks Thunderbolt and compares it with Lightning and SNARK backends. Section 7 surveys related work and Section 8 concludes.

2 Background

This section recalls the cryptographic primitives used by Thunderbolt. All notation introduced here is used without further comment in subsequent sections.

2.1 Elliptic-Curve Groups

Let $\mathbb{G}$ be an elliptic-curve group of prime order q. We fix two independent generators of $\mathbb{G}$: the standard secp256k1 base point G, used for the on-chain lock key $P = (x_s + y_s)\,G$, and a second nothing-up-my-sleeve (NUMS) generator H, used for all holder anchors $X_s = x_s H$ and invoice deltas $D_s = r_s H$. Independence of G and H (i.e., the discrete log of H with respect to G is unknown) is required so that an adversary who learns a holder secret x_s cannot translate it into an opening of the lock key. A *discrete-log pair* is a pair (x, X) with $X = xH$ for some scalar $x \in \mathbb{Z}_q$. Given only X and H, no efficient algorithm can recover x—this is the *elliptic-curve discrete-logarithm problem* (ECDLP), on which the security of Bitcoin and of our proofs rests.

2.2 The Paillier Cryptosystem

The Paillier cryptosystem [32] is an asymmetric encryption scheme whose plaintext space is $\mathbb{Z}_N$ for an RSA modulus $N = pq'$ (we write q' to distinguish the Paillier factor from the curve order q). The public key is (N, g) with $g \in \mathbb{Z}^*_{N^2}$ of order divisible by N. Encryption of a message $m \in \mathbb{Z}_N$ with randomness $\rho \in \mathbb{Z}^*_N$ is

$$\mathrm{Enc}_{pk}(m; \rho) = g^m \rho^N \bmod N^2.$$

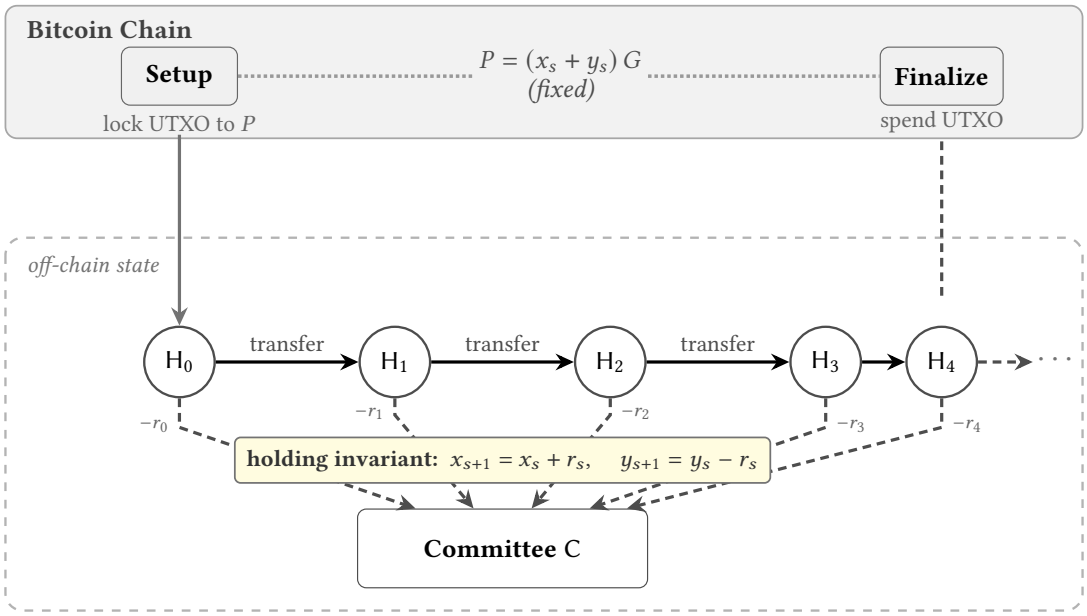


Figure 1: Illustrative overview of Thunderbolt. A Bitcoin UTXO is locked once on-chain under the fixed key $P = (x_s + y_s)G$ at Setup, and the chain sees no further activity until Finalize. Ownership transfers between successive holders $H_0, H_1, \ldots$ happen entirely off-chain (dashed box). At each step, a fresh refresh value r_s is added to the current holder's secret and subtracted from the Committee share, so that $(x_s + y_s)G$ remains equal to P throughout.

Decryption uses the factorization of N. The scheme is semantically secure under the *Decisional Composite Residuosity* (DCR) assumption.

Additive homomorphism. Paillier is *additively* homomorphic: given ciphertexts $c_1 = \mathrm{Enc}(m_1)$ and $c_2 = \mathrm{Enc}(m_2)$, the product

$$c_1 \cdot c_2 \bmod N^2 = \mathrm{Enc}(m_1 + m_2)$$

is a valid encryption of $m_1 + m_2 \bmod N$. We write this operation as $c_1 \oplus c_2$, and the corresponding "ciphertext subtraction" as $c_1 \ominus c_2 := c_1 \cdot c_2^{-1} \bmod N^2$. This additive homomorphism is the reason Thunderbolt can compose a fresh holder ciphertext without decrypting the invoice ciphertext.

2.3 Sigma Protocols and Fiat–Shamir

A *Sigma protocol* [12, 13] is a three-move honest-verifier zero-knowledge proof for a relation $\mathcal{R}$: the prover sends a commitment A, receives a challenge c, and responds with z; the verifier accepts iff a public verification equation on (A, c, z) and the statement holds. The *Fiat–Shamir heuristic* [19] compiles a Sigma protocol into a NIZK in the random-oracle model [3] by setting $c = \mathrm{Hash}(\text{statement}, A)$; soundness follows from the forking lemma [34].

2.4 Verifiable Encryption of Discrete Logarithms

Camenisch and Shoup [10] give an efficient protocol for *verifiable encryption of discrete logarithms*: given a discrete-log pair $(m, M = mH)$ and a Paillier ciphertext $c = \mathrm{Enc}_{pk}(m)$, the prover produces a Sigma-protocol proof that c encrypts the same integer whose discrete log is M. The proof relation is

$$\mathcal{R}_{\mathrm{VE}} = \{(M, c;\ m, \rho) : M = mH \ \wedge\ c = g^m \rho^N \bmod N^2\}.$$

The key technique is a *shared response*: the Sigma-protocol response z simultaneously satisfies both the elliptic-curve verification equation ($zH \stackrel{?}{=} A_{\mathrm{EC}} + cM$) and the Paillier verification equation ($g^z \tau^N \stackrel{?}{=} A_{\mathrm{Pai}} \cdot c^c$), where the same z forces the same underlying integer in both relations. This "shared-z" construction is the technical core of our proofs.

2.5 Verifiable Encrypted Secret Sharing (VESS)

Shoup's *Simple VESS* [40] provides a mechanism for verifiably encrypting a Shamir secret sharing [39] to a committee of n shareholders, building on the line of work initiated by Feldman [18], Pedersen [33], and the distributed key-generation protocols of [21]. The sharing party (the "dealer") publishes encrypted shares $(\hat{s}_1, \ldots, \hat{s}_n)$ together with a proof that there exists a unique polynomial of degree at most $t - 1$ whose evaluations are the plaintexts of these ciphertexts. Any observer can verify the consistency of the encrypted shares without decrypting them. If at least t of the n shareholders are honest, they can collectively reconstruct the shared secret.

In Thunderbolt, VESS is used when the Committee is instantiated as a threshold committee rather than a single actor. The committee-side refresh value r_s is shared via a VESS protocol so that the compensating update $y_{s+1} = y_s - r_s$ is performed collectively, without any single committee member learning y_s or r_s in

the clear. This modular substitution replaces the single-Committee trust assumption with a standard threshold-honesty assumption, and it can be performed without any change to the Invoice and Transfer Consistency Proofs.

3 Design Rationale

This section motivates the combination of the two primitives recalled in Section 2—verifiable encryption of discrete logarithms (VEDL) and verifiable encrypted secret sharing (VESS)—and explains why their composition is both necessary and sufficient for asynchronous off-chain UTXO transfer under a fixed on-chain key.

The binding requirement. A correct transfer step must produce three cryptographic objects simultaneously: a Paillier ciphertext of $x_{s+1} = x_s + r_s$ under Bob's key, a Paillier ciphertext of the refresh r_s under the third party's key, and a public anchor delta $D_s = r_s H$ on the elliptic curve. All three objects must be *bound to the same integer*; otherwise a dishonest party can publish inconsistent values and break the fixed-key invariant $(x_s + y_s) G = P$.

VEDL for per-step correctness. VEDL (Section 2.4) provides exactly this binding. Its shared Sigma response z forces the Paillier plaintext and the elliptic-curve discrete log to be the same integer, without revealing it. We instantiate VEDL in two ways: the Transfer Consistency Proof uses a single-receiver instance to bind the holder secret x_s under X_s to its Paillier ciphertext, while the Invoice Consistency Proof extends VEDL to a multi-receiver setting—a single shared z simultaneously binds one refresh value r_s to the anchor delta D_s and to two Paillier ciphertexts under different keys.

VESS for threshold trust degradation. VEDL alone, however, concentrates the third-party role in a single actor whose compromise exposes the full lock key. VESS (Section 2.5) removes this single point of failure: the refresh r_s is verifiably encrypted as a (t, n)-Shamir sharing, so that any t honest members can reconstruct r_s and perform $y_{s+1} = y_s - r_s$, while no coalition of fewer than t learns anything. This substitution is modular: VESS replaces the single-party ciphertext ct_C with a set of verifiably encrypted shares, and neither the Invoice nor the Transfer Consistency Proof changes form.

Necessity of the composition. Neither primitive alone suffices. VEDL without VESS forces a single trusted third party, concentrating a point of failure that is unacceptable for a payment-channel deployment. VESS without VEDL distributes the third-party role but cannot bind the shared Shamir secret to the on-chain elliptic-curve anchor—an observer would have no way to verify that the anchor delta D_s is consistent with the encrypted refresh shares. Together, VEDL supplies the *per-step correctness guarantee* under a shared integer witness, and VESS supplies *threshold trust degradation* for the third-party role.

4 Cryptographic Primitive

This section develops the two NIZK proofs that certify a single Thunderbolt transfer step, instantiating the VEDL-based construction motivated in Section 3. Throughout, let $\mathbb{G}$ be the secp256k1 group of prime order q with base point H. Let (N_B, g_B) and (N_C, g_C) be Paillier public keys for Bob and the Committee, respectively. All Paillier arithmetic is modulo N_B^2 or N_C^2 as appropriate. All elliptic-curve scalar equalities are taken modulo q.

4.1 Transfer Model

A Thunderbolt state at step s consists of a *holder secret* x_s, a *committee secret* y_s, and a *public anchor* $X_s = x_s H$, subject to the fixed-key invariant

$$P = (x_s + y_s) G. \tag{1}$$

Alice is the current holder; Bob is the next recipient.

A transfer from Alice to Bob proceeds in two phases:

(1) **Invoice phase (Bob).** Bob chooses a fresh refresh value $r_s \in \mathbb{Z}_q$ and publishes:
- a public delta point $D_s = r_s H$,
- a ciphertext to himself, $\mathrm{ct}_B = \mathrm{Enc}_{pk_B}(r_s;\, \rho_B)$,
- a ciphertext to the Committee, $\mathrm{ct}_C = \mathrm{Enc}_{pk_C}(r_s;\, \rho_C)$,
- a proof π_{inv} (the Invoice Consistency Proof) that all three objects encode the same r_s.

(2) **Consume phase (Alice).** Alice forms the next holder ciphertext

$$T_{s+1} = \mathrm{Enc}_{pk_B}(x_s;\, \eta_s) \oplus \mathrm{ct}_B,$$

updates the public anchor to $X_{s+1} = X_s + D_s$, and publishes a proof π_{xfer} (the Transfer Consistency Proof) that she injected the actual current holder secret x_s consistent with X_s.

After the transfer, Bob decrypts T_{s+1} to obtain $x_s + r_s$ and sets $x_{s+1} \equiv x_s + r_s \pmod q$. The Committee recovers r_s from ct_C and updates $y_{s+1} = y_s - r_s$. The fixed-key invariant (1) is preserved:

$$(x_{s+1} + y_{s+1}) G = (x_s + r_s + y_s - r_s) G = (x_s + y_s) G = P.$$

Security assumptions. The security of the Invoice and Transfer Consistency Proofs rests on three standard assumptions:

(1) The *Elliptic-Curve Discrete-Logarithm Problem* (ECDLP) is hard in $\mathbb{G}$: given $X = x H$, no probabilistic polynomial-time (PPT) adversary can recover x with non-negligible probability.
(2) The *Decisional Composite Residuosity* (DCR) assumption [32] holds for the Paillier moduli N_B and N_C: no PPT adversary can distinguish N-th residues in $\mathbb{Z}^*_{N^2}$ from non-residues.
(3) The hash function Hash is modeled as a *random oracle* (ROM) [3], as required by the Fiat–Shamir compilation [19].

Under these assumptions, both proofs are complete, specially sound, and honest-verifier zero-knowledge as interactive Sigma protocols, and non-interactive zero-knowledge in the random-oracle model after Fiat–Shamir compilation. Soundness in the NIZK setting follows from the forking lemma [34].

4.2 Invoice Consistency Proof

Definition 4.1 (Invoice Consistency Relation). *The relation* $\mathcal{R}_{\mathrm{inv}}$ *is defined as*

$$\mathcal{R}_{\mathrm{inv}} = \left\{ (D_s, \mathrm{ct}_B, \mathrm{ct}_C;\ r_s, \rho_B, \rho_C) \ :\ \begin{array}{l} D_s = r_s H, \\ \mathrm{ct}_B = g_B^{r_s}\, \rho_B^{N_B} \bmod N_B^2, \\ \mathrm{ct}_C = g_C^{r_s}\, \rho_C^{N_C} \bmod N_C^2 \end{array} \right\}.$$

Why this proof is needed. Without it, a malicious Bob could commit different refresh values in D_s, ct_B, and ct_C, breaking the fixed-key invariant.

Sigma protocol. The prover (Bob) samples a commitment scalar α and commitment randomnesses β_B, β_C, and computes:

$$A_D = \alpha H,$$
$$A_B = g_B^{\alpha}\,\beta_B^{N_B} \bmod N_B^2,$$
$$A_C = g_C^{\alpha}\,\beta_C^{N_C} \bmod N_C^2.$$

Fiat–Shamir challenge. The challenge is computed as

$$c = \mathsf{Hash}\big(\texttt{tb/invoice},\ \texttt{ctx_id},\ \texttt{inv_id},\ H,\ pk_B,\ pk_C,\ D_s,\ \mathrm{ct}_B,\ \mathrm{ct}_C,\ A_D,\ A_B,\ A_C\big) \bmod q.$$

Response. The prover computes a single shared response and two Paillier randomness responses:

$$z = \alpha + c \cdot r_s,$$
$$\tau_B = \beta_B \cdot \rho_B^{c} \bmod N_B,$$
$$\tau_C = \beta_C \cdot \rho_C^{c} \bmod N_C.$$

The proof is $\pi_{\text{inv}} = (A_D, A_B, A_C, z, \tau_B, \tau_C)$.

Verification. The verifier accepts iff all three of the following hold (same z in all three):

(1) **EC check:** $z\,H \stackrel{?}{=} A_D + c \cdot D_s$.
(2) **Bob–Paillier check:** $g_B^{z}\,\tau_B^{N_B} \stackrel{?}{=} A_B \cdot \mathrm{ct}_B^{c} \pmod{N_B^2}$.
(3) **Committee–Paillier check:** $g_C^{z}\,\tau_C^{N_C} \stackrel{?}{=} A_C \cdot \mathrm{ct}_C^{c} \pmod{N_C^2}$.

Theorem 4.2 (Soundness of the Invoice Consistency Proof). *The Invoice Consistency Proof is a NIZK for $\mathcal{R}_{\text{inv}}$ in the random-oracle model. In particular, the shared response z in all three verification equations forces a single integer r_s to be the discrete log of D_s and the Paillier plaintext of both ct_B and ct_C.*

Proof. We establish the three standard properties.

Completeness. If the prover is honest, then $z = \alpha + c\,r_s$, $\tau_B = \beta_B \cdot \rho_B^{c}$, and $\tau_C = \beta_C \cdot \rho_C^{c}$. For the EC check: $z\,H = (\alpha + c\,r_s)\,H = \alpha\,H + c\,r_s\,H = A_D + c\,D_s$. For the Bob–Paillier check: $g_B^{z}\,\tau_B^{N_B} = g_B^{\alpha + c\,r_s}\,(\beta_B \cdot \rho_B^{c})^{N_B} = (g_B^{\alpha}\,\beta_B^{N_B}) \cdot (g_B^{r_s}\,\rho_B^{N_B})^{c} = A_B \cdot \mathrm{ct}_B^{c}$. The Committee–Paillier check is analogous.

Special soundness. Given two accepting transcripts $(A_D, A_B, A_C,\ c, z, \tau_B, \tau_C)$ and $(A_D, A_B, A_C,\ c', z', \tau_B', \tau_C')$ with $c \neq c'$, define $\Delta c = c - c'$ and $\Delta z = z - z'$. From the EC equation: $\Delta z \cdot H = \Delta c \cdot D_s$, so $D_s = (\Delta z / \Delta c)\,H$, yielding $r_s^{*} = \Delta z / \Delta c \bmod q$. From the Bob–Paillier equation: $g_B^{\Delta z} \cdot (\tau_B / \tau_B')^{N_B} = \mathrm{ct}_B^{\Delta c}$, from which we extract $\rho_B^{*} = (\tau_B / \tau_B')^{(\Delta c)^{-1}} \bmod N_B$ satisfying $\mathrm{ct}_B = g_B^{r_s^{*}} \cdot (\rho_B^{*})^{N_B}$. The Committee–Paillier extraction is identical, yielding ρ_C^{*}. Because a single Δz drives all three extractions, the extracted plaintext r_s^{*} is the same in all three relations, establishing $(D_s, \mathrm{ct}_B, \mathrm{ct}_C;\ r_s^{*}, \rho_B^{*}, \rho_C^{*}) \in \mathcal{R}_{\text{inv}}$.

Honest-verifier zero-knowledge. The simulator $\mathcal{S}$, given the statement $(D_s, \mathrm{ct}_B, \mathrm{ct}_C)$, samples $z \leftarrow \mathbb{Z}_q$, $\tau_B \leftarrow \mathbb{Z}_{N_B}^{*}$, $\tau_C \leftarrow \mathbb{Z}_{N_C}^{*}$, programs the random oracle to return c, and sets: $A_D = z\,H - c\,D_s$, $A_B = g_B^{z}\,\tau_B^{N_B} \cdot \mathrm{ct}_B^{-c} \bmod N_B^2$, $A_C = g_C^{z}\,\tau_C^{N_C} \cdot \mathrm{ct}_C^{-c} \bmod N_C^2$. The simulated transcript $(A_D, A_B, A_C, c, z, \tau_B, \tau_C)$ is identically distributed to an honest transcript, since z is uniform and the commitments are uniquely determined.

Soundness in the non-interactive (Fiat–Shamir compiled) setting follows from the forking lemma [34] applied in the random-oracle model. □

4.3 Transfer Consistency Proof

The Transfer Consistency Proof assumes that the Invoice Consistency Proof has been verified, so the public inputs D_s and ct_B are known to be consistent with a single r_s.

Definition 4.3 (Transfer Consistency Relation). *The relation $\mathcal{R}_{\text{xfer}}$ is defined as*

$$\mathcal{R}_{\text{xfer}} = \left\{ (X_s, \mathrm{ct}_B, T_{s+1};\ x_s, \eta_s) \ :\ \begin{array}{l} X_s = x_s\,H, \\ T_{s+1} \cdot \mathrm{ct}_B^{-1} = g_B^{x_s}\,\eta_s^{N_B} \bmod N_B^2 \end{array} \right\}.$$

Why this proof is needed. Without it, Alice could inject an arbitrary $x_s' \neq x_s$ into T_{s+1}, leaving Bob with a ciphertext inconsistent with the public anchor.

Sigma protocol. The prover (Alice) samples a commitment scalar α and a Paillier commitment randomness β, and computes:

$$A_X = \alpha\,H,$$
$$A_T = g_B^{\alpha}\,\beta^{N_B} \bmod N_B^2.$$

Fiat–Shamir challenge.

$$c = \mathsf{Hash}\big(\texttt{tb/xfer},\ \texttt{ctx_id},\ s{+}1,\ \texttt{inv_id},\ H,\ pk_B,\ X_s,\ \mathrm{ct}_B,\ T_{s+1},\ A_X,\ A_T\big) \bmod q.$$

Response.

$$z = \alpha + c \cdot x_s,$$
$$\tau = \beta \cdot \eta_s^{c} \bmod N_B.$$

The proof is $\pi_{\text{xfer}} = (A_X, A_T, z, \tau)$.

Verification. The verifier accepts iff both of the following hold (same z):

(1) **EC check:** $z\,H \stackrel{?}{=} A_X + c \cdot X_s$.
(2) **Paillier check:** $g_B^{z}\,\tau^{N_B} \stackrel{?}{=} A_T \cdot (T_{s+1} \ominus \mathrm{ct}_B)^{c} \pmod{N_B^2}$.

Additionally, the verifier checks the public anchor transition:

$$X_{s+1} \stackrel{?}{=} X_s + D_s.$$

This check is outside the zero-knowledge relation itself but is necessary for protocol correctness.

Theorem 4.4 (Soundness of the Transfer Consistency Proof). *The Transfer Consistency Proof is a NIZK for $\mathcal{R}_{\text{xfer}}$ in the random-oracle model. The shared response z forces a single integer x_s to be both the discrete log of X_s and the Paillier plaintext of $T_{s+1} \ominus \mathrm{ct}_B$.*

Proof. Let $U = T_{s+1} \ominus \mathrm{ct}_B = T_{s+1} \cdot \mathrm{ct}_B^{-1} \bmod N_B^2$ for brevity.

Completeness. With $z = \alpha + c\,x_s$ and $\tau = \beta \cdot \eta_s^{c}$: $z\,H = \alpha\,H + c\,x_s\,H = A_X + c\,X_s$, and $g_B^{z}\,\tau^{N_B} = g_B^{\alpha}\,\beta^{N_B} \cdot (g_B^{x_s}\,\eta_s^{N_B})^{c} = A_T \cdot U^{c}$.

Special soundness. Given two accepting transcripts with challenges $c \neq c'$, define $\Delta c = c - c'$, $\Delta z = z - z'$. From the EC equation: $x_s^{*} = \Delta z / \Delta c \bmod q$ satisfying $X_s = x_s^{*}\,H$. From the Paillier equation: $\eta_s^{*} = (\tau / \tau')^{(\Delta c)^{-1}} \bmod N_B$ satisfying $U = g_B^{x_s^{*}}\,(\eta_s^{*})^{N_B} \bmod N_B^2$. The shared Δz ensures the same x_s^{*} opens both relations.

Honest-verifier zero-knowledge. The simulator samples $z \leftarrow \mathbb{Z}_q$, $\tau \leftarrow \mathbb{Z}^*_{N_B}$, programs c via the random oracle, and sets $A_X = z\,H - c\,X_s$ and $A_T = g_B^z\,\tau^{N_B} \cdot U^{-c} \bmod N_B^2$. The resulting transcript is identically distributed to an honest one.

As with the Invoice proof, non-interactive soundness follows from the forking lemma [34]. □

4.4 Joint Consequence: Correct Transfer Step

Theorem 4.5 (Transfer Correctness). *If both π_{inv} and π_{xfer} verify, and the anchor transition $X_{s+1} = X_s + D_s$ holds, then there exist integers x_s and r_s such that*

$$X_s = x_s\,H, \tag{2}$$

$$D_s = r_s\,H, \tag{3}$$

$$T_{s+1} = \mathrm{Enc}_{pk_B}(x_s + r_s), \tag{4}$$

$$X_{s+1} = (x_s + r_s)\,H. \tag{5}$$

Furthermore, the committee-side update $y_{s+1} = y_s - r_s$ preserves the invariant (1):

$$(x_{s+1} + y_{s+1})\,G = (x_s + y_s)\,G = P.$$

Proof. From the Invoice Consistency Proof (Theorem 4.2), extract r_s satisfying $D_s = r_s\,H$ and $\mathrm{ct}_B = \mathrm{Enc}_{pk_B}(r_s)$. From the Transfer Consistency Proof (Theorem 4.4), extract x_s satisfying $X_s = x_s\,H$ and $T_{s+1} \ominus \mathrm{ct}_B = \mathrm{Enc}_{pk_B}(x_s)$. By Paillier's additive homomorphism, $T_{s+1} = \mathrm{Enc}_{pk_B}(x_s) \oplus \mathrm{Enc}_{pk_B}(r_s) = \mathrm{Enc}_{pk_B}(x_s + r_s)$, giving (4). The anchor transition gives $X_{s+1} = X_s + D_s = x_s\,H + r_s\,H = (x_s + r_s)\,H$, giving (5). Setting $x_{s+1} \equiv x_s + r_s \pmod q$ and $y_{s+1} \equiv y_s - r_s \pmod q$, we obtain $(x_{s+1} + y_{s+1})\,G = (x_s + y_s)\,G = P$. □

4.5 Threshold Committee via Simple VESS

The construction presented so far assumes a *single* third party C that holds the committee share y_s and receives the refresh ciphertext $\mathrm{ct}_C = \mathrm{Enc}_{pk_C}(r_s)$. This is a strong trust assumption: a compromised C can collude with any holder to recover the full lock key $x_s + y_s$ and steal the UTXO. In this subsection we show how to replace C with a threshold committee of n members, of which any t suffice to perform the compensating update, while no coalition of fewer than t learns anything about y_s or r_s. The key observation is that this upgrade is a *modular substitution*: we replace the single Paillier ciphertext ct_C with a Simple VESS distribution [40], and the Invoice and Transfer Consistency Proofs remain unchanged.

Setup. Let the committee consist of n members $\mathrm{C}_1, \ldots, \mathrm{C}_n$, each holding a Paillier key pair (pk_i, sk_i). At channel open, the initial committee share y_0 is distributed via a (t, n)-Shamir secret sharing: each member C_i receives a share $y_0^{(i)}$ such that any t shares reconstruct y_0, and no set of fewer than t shares reveals any information about y_0. The public verification key for the sharing is the set of commitments $\{Y_0^{(i)} = y_0^{(i)}\,G\}_{i\in[n]}$, which allows any observer to verify individual share consistency against the polynomial.

Invoice phase modification. When Bob issues an invoice at step s, he replaces the single ciphertext $\mathrm{ct}_C = \mathrm{Enc}_{pk_C}(r_s)$ with a VESS distribution of r_s:

(1) Bob samples a random degree-$(t-1)$ polynomial $f(\cdot)$ over $\mathbb{Z}_q$ with $f(0) = r_s$.

(2) For each committee member C_i, Bob computes the share $r_s^{(i)} = f(i)$ and its encryption $\hat{r}_s^{(i)} = \mathrm{Enc}_{pk_i}(r_s^{(i)};\, \rho_i)$.

(3) Bob publishes the encrypted shares $(\hat{r}_s^{(1)}, \ldots, \hat{r}_s^{(n)})$ together with polynomial commitments $\{F_j = a_j\,H\}_{j=0}^{t-1}$, where $a_0, \ldots, a_{t-1}$ are the coefficients of f (so $F_0 = r_s\,H = D_s$).

(4) Bob proves, via the Simple VESS verification equations [40], that the encrypted shares are consistent with the committed polynomial. Concretely, each observer checks that $\hat{r}_s^{(i)}$ encrypts a value whose discrete-log image under H matches the evaluation of the committed polynomial at i: $\sum_{j=0}^{t-1} i^j \cdot F_j$.

The Invoice Consistency Proof π_{inv} is unchanged: it still binds r_s to the anchor delta $D_s = r_s\,H$ and to Bob's ciphertext ct_B. The VESS distribution provides an *independent, publicly verifiable* guarantee that the same r_s is correctly shared among the committee members. The binding between these two guarantees is the shared free term: $F_0 = r_s\,H = D_s$, which appears in both the Invoice Consistency Proof and the VESS polynomial commitment.

Committee update. Upon receiving the VESS distribution, each committee member C_i decrypts $\hat{r}_s^{(i)}$ to recover $r_s^{(i)}$ and updates its local share of the committee secret:

$$y_{s+1}^{(i)} = y_s^{(i)} - r_s^{(i)}.$$

This update is performed locally by each member without communication. No member learns r_s itself; each member only sees its own share $r_s^{(i)}$.

Reconstruction and finalization. When the current holder wishes to finalize the channel (i.e., spend the UTXO on-chain), she needs the committee to contribute its share of the signing key. Any t committee members can jointly reconstruct y_s via Lagrange interpolation on their shares $\{y_s^{(i)}\}_{i\in S}$ for any subset $S \subseteq [n]$ with $|S| \geq t$. The resulting y_s is combined with the holder's x_s to produce a threshold signature under $P = (x_s + y_s)\,G$ using a standard threshold Schnorr [25] or ECDSA [26] protocol.

Correctness. The share update preserves the Shamir structure. At step s, define the committee polynomial $g_s(\cdot)$ by $g_s(i) = y_s^{(i)}$. At step $s+1$:

$$g_{s+1}(i) = y_s^{(i)} - r_s^{(i)} = g_s(i) - f(i),$$

where f is the degree-$(t-1)$ sharing polynomial for r_s. Since both g_s and f are degree-$(t-1)$ polynomials, $g_{s+1} = g_s - f$ is also degree-$(t-1)$, and its free term is $y_s - r_s = y_{s+1}$. Thus the shares $\{y_{s+1}^{(i)}\}$ form a valid (t, n)-Shamir sharing of y_{s+1}, and the fixed-key invariant $(x_{s+1} + y_{s+1})\,G = P$ is preserved.

Security. The threshold committee instantiation inherits its security from two independent guarantees:

- **Secrecy of y_s:** As long as fewer than t committee members are corrupted, the adversary obtains fewer than t shares of y_s and learns nothing about the committee secret (by the information-theoretic security of Shamir sharing).
- **Consistency of r_s:** The VESS verification equations ensure that a malicious Bob cannot distribute inconsistent shares; any observer can verify that the encrypted shares are consistent with a single degree-$(t-1)$ polynomial whose free term

matches D_s. If the VESS verification passes but the shares are inconsistent, the observer detects this and rejects the invoice.

The resulting trust assumption is the standard (t, n)-threshold honesty assumption: at most $t-1$ of the n committee members may be corrupted. This is the same assumption used by every modern threshold signing deployment (e.g., FROST [25], ROAST [37]), and it replaces the single-point-of-failure assumption of the basic construction with a well-understood distributed trust model.

Theorem 4.6 (VESS Modularity). *Let Π_{single} denote the single-third-party Thunderbolt construction of Section 4.1–Section 4.4, and let Π_{thr} denote the threshold-committee instantiation of this subsection. Then Π_{thr} preserves every property of Π_{single}—fixed-key invariance, anchor correctness, ciphertext binding, and soundness of the Invoice and Transfer Consistency Proofs—under the assumption that at most $t-1$ of the n committee members are corrupted and that the Simple VESS distribution of [40] is secure.*

Proof. The Invoice and Transfer Consistency Proofs depend only on the public objects D_s, ct_B, X_s, and T_{s+1}, none of which are changed by the VESS substitution. Hence π_{inv} and π_{xfer} remain sound and zero-knowledge as before. Correctness of the committee-side update follows from the Shamir-preservation argument above: the shares $\{y_{s+1}^{(i)}\}_{i\in[n]}$ form a valid (t, n)-sharing of $y_{s+1} = y_s - r_s$ with free term matching $F_0 = r_s H = D_s$. Secrecy of y_s and r_s holds information-theoretically as long as fewer than t members are corrupted. Finally, VESS verification [40] ensures that a malicious Bob cannot distribute inconsistent shares undetected. □

5 Thunderbolt as a Payment Channel

Having established the core cryptographic primitive in Section 4, we now formalize Thunderbolt as a *payment channel*—a long-lived off-chain object that is anchored by a fixed on-chain UTXO and evolves through a sequence of off-chain state updates. This section introduces the participants, the channel lifecycle, and the key state invariants. We deliberately use the same terminology as the classical payment-channel literature [15, 35] so that the differences are easy to see.

5.1 Participants and Setup

A Thunderbolt channel has three participant roles:

- The **Holder** (initially H_0) is the party that currently owns the UTXO off-chain. Over the lifetime of the channel, the holder role is handed from one user to the next: $\mathsf{H}_0 \to \mathsf{H}_1 \to \mathsf{H}_2 \to \cdots$, where each H_s is an independent user.
- The **Committee** C is a fixed (but possibly threshold) party that maintains the complementary share y_s of the lock key. C never owns the UTXO and never learns the holder secret x_s; its sole role is to preserve the fixed-key invariant (1) via the compensating "subtract" update.
- The **Chain** plays the role of a passive verifier: it sees exactly two transactions over the channel's lifetime (Setup and Finalize), regardless of how many holder transitions occur in between.

5.2 Threat Model

We consider a static-corruption adversary $\mathcal{A}$ that may corrupt any subset of holders and at most $t-1$ of the n committee members before the channel opens. The adversary is PPT and interacts with the protocol in the random-oracle model.

Security guarantees. If fewer than t committee members are corrupted and all proofs $(\pi_{\text{inv}}, \pi_{\text{xfer}})$ verify, the protocol preserves three invariants at every step s:

- *Fixed-key integrity:* $(x_s + y_s)\,G = P$. No adversary can cause the on-chain key to change without an on-chain transaction.
- *Ownership authenticity:* Only the holder who knows x_s can produce a valid Transfer Consistency Proof for step s. In particular, a corrupted prior holder $\mathsf{H}_{s'}$ (with $s' < s$) cannot produce a valid proof for the current state.
- *Ciphertext binding:* The next holder's ciphertext T_{s+1} encrypts $x_s + r_s$ under Bob's Paillier key and no other value, as guaranteed by the soundness of both proofs jointly (Theorem 4.5).

Non-goals. The protocol does *not* guarantee:

- *Liveness:* If all n committee members go offline, the holder cannot finalize the channel. Liveness requires at least t responsive committee members.
- *Transfer-graph privacy:* Invoices and transfer proofs are published to a shared off-chain log visible to all participants. An observer can trace the sequence of holder transitions. Concealing the transfer graph is an orthogonal problem left to future work.

Setup (on-chain). At channel open, the initial holder H_0 and the Committee jointly derive the fixed public key $P = (x_0 + y_0)\,G$, where x_0 is H_0's initial holder secret and y_0 is the Committee's initial share. H_0 funds a UTXO on the Bitcoin chain locked to a script whose spending condition enforces knowledge of the discrete log of P (e.g., a standard BIP340 Taproot output [43]). This is the *only* on-chain operation at open. The initial public anchor is $X_0 = x_0 H$, which is stored in the channel's off-chain state.

Off-chain state at step s. The channel state at step $s \geq 0$ consists of:

- the public anchor X_s,
- the holder ciphertext T_s (a Paillier ciphertext under the current holder's public key, encrypting the holder secret x_s),
- the committee share y_s (held by C; in a threshold committee, held collectively via Shamir secret sharing),
- the chain of transfer proofs $\{(\pi_{\text{inv}}^{(s')}, \pi_{\text{xfer}}^{(s')})\}_{s'\leq s}$.

The fixed-key invariant $(x_s + y_s)\,G = P$ holds at every step by Theorem 4.5.

5.3 Channel Lifecycle

A Thunderbolt channel passes through three phases: **Open**, **Transfer** (zero or more times), and **Finalize**.

Open. H_0 and C jointly generate (x_0, y_0) and publish the on-chain UTXO under $P = (x_0 + y_0)\,G$. H_0 initializes $T_0 = \mathrm{Enc}_{pk_{\mathsf{H}_0}}(x_0)$, $X_0 = x_0 H$.

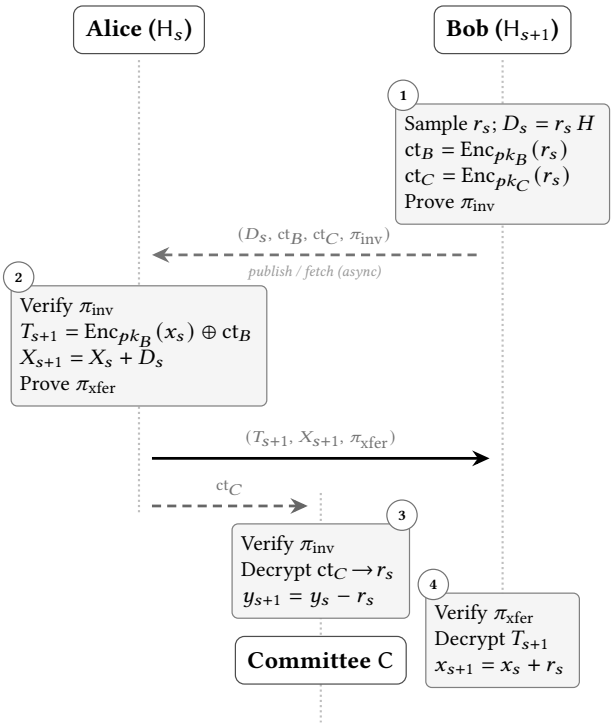


Figure 2: Transfer phase detail (one step $s \to s{+}1$). Bob creates an invoice with π_{inv} (①); Alice verifies π_{inv}, consumes the invoice, and publishes the updated ciphertext with π_{xfer} (②); the Committee verifies π_{inv} and applies the compensating subtract (③); Bob verifies π_{xfer} and decrypts the new holder secret (④). Steps ③ and ④ proceed independently.

Transfer. To transfer ownership from H_s (Alice) to H_{s+1} (Bob):

(1) Bob chooses a fresh refresh value r_s, computes $D_s = r_s H$, $ct_B = Enc_{pk_B}(r_s)$, $ct_C = Enc_{pk_C}(r_s)$, and produces the Invoice Consistency Proof π_{inv}. Bob publishes the invoice tuple $(D_s, ct_B, ct_C, \pi_{inv})$.
(2) Alice fetches the invoice, verifies π_{inv}, computes $T_{s+1} = Enc_{pk_B}(x_s) \oplus ct_B$, the new anchor $X_{s+1} = X_s + D_s$, and produces the Transfer Consistency Proof π_{xfer}. She publishes the transfer tuple $(T_{s+1}, X_{s+1}, \pi_{xfer})$. Alice need not contact Bob directly.
(3) The Committee verifies π_{inv}, decrypts ct_C to recover r_s, and updates its share to $y_{s+1} = y_s - r_s$.
(4) Bob verifies π_{xfer} and decrypts T_{s+1} to recover $x_{s+1} = x_s + r_s$.

No step in this phase requires Alice and Bob to be online simultaneously. No step touches the Bitcoin chain. Figure 2 illustrates the complete transfer phase.

Finalize. When the current holder H_s wishes to withdraw on-chain, she and C jointly produce a signature on the closing transaction under $P = (x_s + y_s)\,G$, using any standard two-party ECDSA or Schnorr protocol [25, 26]. Because the on-chain key has not changed since Open, the Bitcoin node verifies the signature exactly as it would any other Taproot spend.

5.4 Key Invariants

The Thunderbolt protocol preserves the following invariants at every step s, assuming all transfer proofs verify:

(1) **Fixed-key invariant:** $(x_s + y_s)\,G = P$. The on-chain lock key never changes.
(2) **Anchor correctness:** $X_s = x_s H$. The public anchor is the discrete-log image of the current holder secret.
(3) **Ciphertext correctness:** $T_s = Enc_{pk_{H_s}}(x_s)$. The current holder ciphertext encrypts the current holder secret under the current holder's key.
(4) **Proof chain:** For every step $s' < s$, both $\pi_{inv}^{(s')}$ and $\pi_{xfer}^{(s')}$ verify.

Invariants 1–3 follow from Theorem 4.5 by induction on s, and invariant 4 is a public-check by any verifier.

Comparison with Lightning. Thunderbolt and Lightning both anchor an off-chain channel to an on-chain UTXO, but differ fundamentally: Lightning requires a routed graph of pre-funded, bidirectionally-online channels with directional liquidity, while Thunderbolt needs no graph, no routing, no rebalancing, and is fully asynchronous. A detailed quantitative comparison appears in Section 6.2.

5.5 Generality Beyond Bitcoin

Although we present Thunderbolt in Bitcoin language, its cryptographic core is not Bitcoin-specific. The Invoice and Transfer Consistency Proofs depend only on (i) a prime-order group $\mathbb{G}$ with a hard discrete-logarithm problem, (ii) an additively homomorphic encryption scheme, and (iii) a hash function modeled as a random oracle. Bitcoin enters the picture only at Setup (committing the fixed key P to a UTXO) and Finalize (signing under P)—both are instantiations of a generic "fixed-public-key anchoring" interface realizable by any system with a non-repudiable public-key commitment.

Alternative instantiations.

- *Ethereum and EVM chains.* The anchoring interface becomes a smart contract holding P; Finalize is a contract method gated by a signature under P. The off-chain transfer mechanics are unchanged.
- *Threshold custody rotation.* Institutional custodians can hand off the "active signer" role among sub-custodians without rotating the public key visible to external counterparties.
- *Off-chain credential delegation.* Any fixed-public-key authentication system (FIDO, DID documents) can delegate signing authority through a sequence of holders without re-registering the key with the relying party.

We adopt Bitcoin as the principal instantiation because it is the most constrained setting—a stateless UTXO model with minimal scripting. A protocol that works under Bitcoin's constraints automatically works in richer environments.

6 Evaluation

We evaluate Thunderbolt along three axes: (1) per-stage microbenchmarks of a single off-chain transfer (Section 6.1), (2)

Stage	Avg (ms)	Min (ms)	Max (ms)
Invoice setup	118.1	114.4	143.2
Invoice prove	305.3	294.5	351.3
Invoice verify	194.1	187.0	232.9
Transfer setup	63.3	60.9	68.1
Transfer prove	159.3	154.3	178.6
Anchor verify	<0.01	<0.01	<0.01
Transfer verify	109.0	105.5	117.5
Bob decrypt	30.2	28.6	34.0
Committee update	30.2	29.1	34.8
Total (off-chain)	**1009.6**	**978.6**	**1089.9**

Table 1: Per-stage wall-clock time for a single Thunderbolt off-chain transfer (100 iterations, 3072-bit Paillier modulus, secp256k1).

Category	Time (ms)	Share (%)
ZKP proving (Invoice + Transfer)	464.6	46.0
ZKP verification (Invoice + Transfer)	303.1	30.0
Setup (Invoice + Transfer)	181.4	18.0
Auxiliary (anchor, decrypt, committee)	60.4	6.0
Total	**1009.6**	**100**

Table 2: Cost breakdown by functional category.

a system-level comparison against the Lightning Network (Section 6.2), and (3) a comparison of our Sigma-protocol construction against general-purpose SNARK backends (Section 6.3).

Experimental setup. All benchmarks were run on a single machine with an Apple M4 processor (10-core, 4 performance + 6 efficiency), 16 GB unified memory, macOS 15.3, compiled with `rustc` 1.87-nightly in release mode with LTO enabled. The Thunderbolt implementation uses the `rust-paillier` crate for Paillier operations with a 3072-bit modulus ($|N| = 3072$, so all Paillier arithmetic operates over 6144-bit integers in $\mathbb{Z}_{N^2}$) and the `k256` crate for secp256k1 elliptic-curve arithmetic. Each experiment reports the average, minimum, and maximum over 100 iterations, with a 3-iteration warmup discarded before measurement.

6.1 Per-Stage Microbenchmark

A single off-chain transfer executes nine stages in sequence: invoice setup, invoice prove, invoice verify, transfer setup, transfer prove, anchor verify, transfer verify, Bob decrypt, and committee update. Table 1 reports the measured wall-clock time for each stage.

Cost breakdown. The total off-chain transfer completes in ≈ 1.01 **s** on average. Table 2 groups the nine stages by their functional role: proving, verification, setup, and auxiliary operations.

The two ZKP proving steps dominate at 46% of the total: Invoice Prove (305 ms) accounts for three Paillier exponentiations in $\mathbb{Z}_{N^2}$ (two commitment exponentiations plus one EC scalar multiplication), while Transfer Prove (159 ms) requires two (one Paillier, one EC). Verification is the second-largest cost at 30%, with a similar structure but operating on the verification equations rather than generating commitments. Setup operations (key derivation, Paillier encryption of the refresh value and holder secret) contribute 18%. The remaining 6% consists of the anchor-point verification ($< 10\,\mu$s, a single EC point comparison), Bob's Paillier decryption (30 ms), and the committee's share update (30 ms, also a single Paillier decryption).

Bottleneck analysis. The dominant cost is modular exponentiation in $\mathbb{Z}_{N^2}$ ($|N| = 3072$, so 6144-bit integers). EC operations (256-bit secp256k1) are roughly $10^3\times$ faster and contribute negligibly. This Paillier-heavy cost profile is why circuit-based backends, which must express these exponentiations as field-arithmetic constraints, pay orders-of-magnitude overhead.

Proof size. Each Invoice Consistency Proof π_{inv} consists of three commitments ($A_D \in \mathbb{G}$, $A_B, A_C \in \mathbb{Z}_{N^2}$), one shared response $z \in \mathbb{Z}_q$, and two randomness responses $\tau_B, \tau_C \in \mathbb{Z}_N$, totaling ≈ 2.3 KB at $|N| = 3072$. Each Transfer Consistency Proof π_{xfer} totals ≈ 1.5 KB. The combined proof payload per transfer step is therefore ≈ 3.8 KB, small enough for a single TCP segment. A channel with m transfers accumulates $\approx 3.8m$ KB of off-chain state (≈ 3.7 MB at 1000 transfers).

Dimension	Lightning	Thunderbolt
Topology	Routed graph	Per-UTXO holder chain
Connectivity	Path-reachable	No path required
Routing	Onion-routed HTLCs	No routing
Liquidity	Directional / channel	Entire UTXO moves
Rebalancing	Required	Not applicable
Online assumption	Both endpoints	Asynchronous
On-chain / channel	2 tx (open, close)	2 tx (open, close)
On-chain / transfer	0 (amortized)	0 (amortized)
Holders / channel	2 (fixed endpoints)	Unbounded chain
Watchtowers	Required for safety	Committee (always-on)
Trust model	Trustless + WT	(t, n) VESS

Table 3: System-level comparison between Lightning and Thunderbolt.

6.2 System-Level Comparison: Thunderbolt vs. Lightning

Table 3 summarizes the structural differences between Thunderbolt and the Lightning Network along the dimensions that matter for real-world deployment.

Table 3 summarizes the structural differences between Thunderbolt and the Lightning Network along the dimensions that matter for real-world deployment.

Connectivity and routing. Lightning's most binding constraint is that every payment requires a path in the channel graph; a payee with no inbound liquidity is unreachable. Thunderbolt eliminates this constraint entirely: a transfer from H_s to H_{s+1} requires only that H_{s+1} publishes an invoice.

Liquidity. A Lightning channel's capacity is directional: funds pushed one way cannot flow back without rebalancing via on-chain or loop-out operations. In Thunderbolt, the entire UTXO transfers at once, so there is no directional capacity and no rebalancing cost.

Asynchrony. Lightning HTLCs require both sender and receiver to be online for the duration of the payment. Thunderbolt transfers are asynchronous: Bob posts his invoice once; Alice consumes it whenever she chooses. This makes Thunderbolt suitable for payroll batches, cold-wallet receipts, and low-power devices that wake up only periodically.

Latency comparison. Measured Lightning payment latencies vary widely by route length and network conditions. Using c-lightning probe-plugin data as reference: a 4-hop round-trip takes ≈ 2.56 s, and the median payment completion time is ≈ 13.1 s (including path-finding and retries). Thunderbolt completes a transfer in ≈ 1.01 s, yielding **2.5×–13×** improvements. Crucially, Thunderbolt's latency is *deterministic*—it depends on local Paillier computation, not network hops or failed routes.

Trust and on-chain scope. Lightning is trustless modulo watchtowers. Thunderbolt relies on the Committee; the threshold-committee instantiation (Section 4.5) reduces this to standard (t, n)-threshold honesty. Both systems amortize on-chain cost, but Lightning amortizes per *channel pair* ($\Theta(n)$ on-chain operations for n users) while Thunderbolt amortizes per *UTXO* ($O(1)$).

6.3 Sigma Protocol vs. General-Purpose SNARKs

An alternative realization of the Invoice and Transfer relations would encode them as arithmetic circuits and feed them into a general-purpose SNARK backend (e.g., a zkVM such as Succinct's SP1 [42], RISC Zero [36], or a circuit-based backend such as Groth16 [22] or Plonk [20]). We argue—and partially measure—that this path is orders of magnitude more expensive than our direct Sigma-protocol instantiation.

Cost drivers. The Invoice and Transfer relations involve two kinds of expensive operations:

- **Paillier arithmetic:** modular exponentiation in $\mathbb{Z}_{N^2}$ for a 3072-bit N, which in a SNARK circuit requires bit-decomposition of 6144-bit integers and thousands of field operations per exponentiation.
- **Elliptic-curve scalar multiplication:** secp256k1 scalar multiplication, requiring 256 doublings and up to 256 additions; each EC operation in a SNARK circuit is a non-trivial sub-circuit over a non-native field.

In a general-purpose circuit backend, each Paillier ciphertext verification alone consumes on the order of 10^5–10^6 constraints, and each EC operation adds 10^3–10^4 more. The resulting circuit size is dominated by Paillier exponentiations, making this one of the worst-case workloads for generic SNARK backends.

Sigma-protocol cost. Our direct Sigma-protocol construction requires no circuit. Each proof consists of a constant number of native group operations:

- Invoice Consistency Proof: 3 commitments (1 EC, 2 Paillier), 1 Fiat–Shamir hash, 1 shared response, 2 Paillier randomness responses. Verification: 3 equations (1 EC, 2 Paillier).
- Transfer Consistency Proof: 2 commitments (1 EC, 1 Paillier), 1 hash, 1 response, 1 Paillier randomness response. Verification: 2 equations (1 EC, 1 Paillier).

Backend	Prove	Verify	Setup
Thunderbolt (Sigma + FS)	465 ms	303 ms	None
SP1 zkVM (u64 only)	$\gg$ 465 ms	$\gg$ 303 ms	None
Groth16 (projected)	~min	~3 ms	Trusted

Table 4: Proving and verification cost for one complete transfer step (Invoice + Transfer). SP1 row uses simplified u64 arithmetic (optimistic lower bound). Groth16 row is projected from published Paillier-in-circuit estimates.

Concretely, proving both proofs takes ≈ 465 ms and verifying both takes ≈ 303 ms, with no constraint system, no trusted setup, and no circuit-level tooling. Table 4 summarizes the comparison.

SP1 benchmark methodology. To provide a concrete reference for the SNARK approach, we implemented the same transfer semantics (holder-secret update $x_{s+1} = x_s + r_s$, committee update $y_{s+1} = y_s - r_s$, invariant check) inside Succinct's SP1 zkVM [42]. Note that this SP1 program uses native u64 arithmetic, *not* the full 3072-bit Paillier operations from our real protocol. It therefore represents an *optimistic lower bound* on what a zkVM would cost for the full relation: the real SNARK circuit would additionally need to express Paillier modular exponentiation over 6144-bit integers, which is the dominant cost in our native benchmarks.

Circuit complexity estimate. The Invoice proof alone would exceed 10^6 R1CS constraints ($\approx 2 \times 10^4$ for EC scalar multiplication + $\approx 3 \times 10^5$–10^6 per Paillier exponentiation over 6144-bit integers). At published Groth16 proving rates of 10^4–10^5 constraints/s, this yields tens of seconds to minutes—a 100–1000× overhead over our 305 ms native prove. The zkVM path adds a further constant from RISC-V emulation.

Discussion. Groth16 proofs are smaller (288 bytes vs. our $\approx$ 3.8 KB) but require a per-circuit trusted setup; STARK-based backends are transparent but produce 50–200 KB proofs. Our relations are *algebraic* (DL + Paillier equalities with a shared witness), precisely the regime where Sigma protocols dominate circuit-based alternatives. Thunderbolt trades a modestly larger proof for orders-of-magnitude lower proving cost and no trusted setup.

7 Related Work

Payment channels and layer-2 protocols. The concept of off-chain payment channels dates to Spilman [41] and was substantially generalized by Poon and Dryja's Lightning Network [35], which chains channels into a routed graph using HTLCs. Decker and Wattenhofer [15] propose duplex micropayment channels using timelocked commitment trees. The *eltoo* (ANYPREVOUT) update mechanism [14] simplifies channel state management at the protocol level but still assumes a bidirectional two-party channel. Broader surveys [23] catalogue the full space of layer-two designs, and concurrent work has extended channel networks with privacy and concurrency guarantees [27], atomic multi-channel updates [17], off-chain rebalancing [24], and faster state-channel state machines [29]. All of these designs require the endpoints of a channel to be fixed at open time and to be online during payment. Thunderbolt differs by allowing the "endpoint" to change: the holder role moves from

one user to the next via off-chain transfer, without opening a new channel.

Virtual channels and channel factories. Dziembowski et al. [16] introduce virtual channels that avoid the need for a direct link between payer and payee by tunneling through an intermediary's channels. Burchert et al. [8] propose channel factories that batch-open many channels in a single on-chain transaction. Aumayr et al. [2] adapt the virtual-channel idea to Bitcoin's limited scripting environment. These reduce per-channel on-chain cost but still require channels between specific pairs; Thunderbolt eliminates per-pair channels entirely via a single UTXO lock supporting unbounded holder transitions.

Verifiable encryption of discrete logarithms. Camenisch and Shoup [10] give an efficient Sigma-protocol [12, 13] proof that a Paillier ciphertext encrypts the discrete log of a known group element, now standard in threshold cryptography, anonymous credentials [9], voting, and escrow. Asokan et al. [1] give earlier constructions for verifiable encryption in the context of optimistic fair exchange. Our Invoice and Transfer Consistency Proofs are direct instantiations of Camenisch–Shoup verifiable encryption, specialized to the "one add, one subtract" invariant of Thunderbolt.

Verifiable encrypted secret sharing. Shoup's Simple VESS [40] provides a verifiable encryption of Shamir secret sharing [39] to a committee, designed as a building block for distributed key generation [21] and resharing. Earlier work on verifiable secret sharing includes Feldman [18] and Pedersen [33], which provide information-theoretic verification but not encryption to specific recipients. Thunderbolt uses VESS when the Committee is instantiated as a threshold group, to distribute the committee-side refresh r_s so that no single member learns the committee share y_s.

Threshold signatures for Bitcoin. FROST [25] and ROAST [37] provide efficient threshold Schnorr [38] signing, which can be used to finalize a Thunderbolt channel when the on-chain output is a Taproot [44] key. Lindell [26] gives a two-party ECDSA protocol, and Canetti et al. [11] a proactive threshold-ECDSA protocol with UC security. Compact multi-signatures [6] offer a complementary route to reducing on-chain key material. These schemes are complementary to Thunderbolt: they handle the on-chain finalization, while our Sigma-protocol proofs handle the off-chain state transitions.

General-purpose zero-knowledge proofs. Groth16 [22], Plonk [20], transparent STARKs [4], and zkVM-based systems such as Succinct SP1 [42] and RISC Zero [36] can in principle prove any NP statement. While powerful, these systems pay a large constant for algebraic relations that Sigma protocols handle natively. Recursive proof composition without trusted setup [7] and accumulation-based aggregation [5] reduce verification costs at scale but do not eliminate the proving-time overhead for Paillier-heavy statements.

Off-chain protocols beyond Bitcoin. Ethereum's layer-2 landscape includes optimistic rollups [31] and zk-rollups [28], which batch many transactions into a single on-chain proof or challenge. These systems benefit from Ethereum's expressive smart contracts and are not directly comparable to Bitcoin-native protocols. Thunderbolt is designed specifically for Bitcoin's constrained scripting environment.

8 Conclusion

We presented Thunderbolt, an off-chain Bitcoin protocol that enables asynchronous and recursive transfer of UTXO ownership between uncoordinated users, while keeping the on-chain lock key fixed throughout the channel's lifetime. Thunderbolt is built from two well-studied cryptographic primitives—Camenisch–Shoup verifiable encryption of discrete logarithms and Shoup's Simple VESS—assembled in a new way to satisfy the "one add, one subtract" invariant that preserves the fixed on-chain key under a sequence of holder transitions.

The technical core of Thunderbolt is a pair of compact NIZK proofs. The *Invoice Consistency Proof* certifies that a recipient's public delta point and two Paillier ciphertexts (to the recipient and to the Committee) encode the same refresh value r_s, using a single shared Sigma-protocol response to bind three different relations to the same integer. The *Transfer Consistency Proof* certifies that the current holder injects the actual holder secret x_s—the discrete log of the public anchor X_s—into the homomorphically updated ciphertext for the next recipient. Together with the elliptic-curve anchor transition $X_{s+1} = X_s + D_s$, these two proofs implement a single correct transfer step without touching the Bitcoin chain.

What Thunderbolt buys. Unlike Lightning, Thunderbolt eliminates pre-funded channel-graph routing, directional-liquidity and rebalancing costs, and the simultaneous-online requirement for sender and receiver. Compared with a general-purpose SNARK realization of the same statements, the Sigma-protocol approach avoids circuit-level tooling entirely and achieves proving and verification costs orders of magnitude smaller on the same relations, because Paillier and elliptic-curve operations are handled natively rather than simulated inside an arithmetic circuit.

Design philosophy. The contribution of this paper is deliberately *not* a new cryptographic assumption. Both primitives we rely on have been scrutinized for over two decades and are already deployed in threshold-signing, escrow, and distributed key-generation systems. What we contribute is the identification of a new *usage scenario* for these reliable primitives: asynchronous, recursive, off-chain Bitcoin ownership transfer under a fixed on-chain footprint. We believe this illustrates a broader point—that well-understood cryptographic building blocks, when combined with the right invariant, can unlock protocol capabilities that would otherwise seem to require novel cryptography.

Future directions. Several directions warrant further exploration. First, a full formal analysis of Thunderbolt under the Universal Composability framework would strengthen the protocol-level guarantees beyond the proof-level soundness and zero-knowledge we rely on here. Second, integrating VESS-based resharing into the Committee layer and quantifying end-to-end threshold-committee costs remain future work. Third, a routed *graph* of Thunderbolt channels—analogous to Lightning's graph but with asynchronous endpoints—could combine the best properties of both systems. We leave these extensions to future work.

References

[1] N. Asokan, Victor Shoup, and Michael Waidner. 1998. Optimistic Fair Exchange of Digital Signatures. In *Advances in Cryptology — EUROCRYPT '98 (LNCS, Vol. 1403)*. Springer, 591–606. doi:10.1007/BFb0054156

[2] Lukas Aumayr, Oguzhan Ersoy, Andreas Erwig, Sebastian Faust, Kristina Hostáková, Matteo Maffei, Pedro Moreno-Sanchez, and Siavash Riahi. 2021. Bitcoin-Compatible Virtual Channels. In *2021 IEEE Symposium on Security and Privacy (S&P)*. IEEE, 901–918. doi:10.1109/SP40001.2021.00097

[3] Mihir Bellare and Phillip Rogaway. 1993. Random Oracles Are Practical: A Paradigm for Designing Efficient Protocols. In *Proceedings of the 1st ACM Conference on Computer and Communications Security (CCS)*. ACM, 62–73. doi:10.1145/168588.168596

[4] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. 2019. Scalable Zero Knowledge with No Trusted Setup. In *Advances in Cryptology — CRYPTO 2019 (LNCS, Vol. 11694)*. Springer, 701–732. doi:10.1007/978-3-030-26954-8_23

[5] Dan Boneh, Justin Drake, Ben Fisch, and Ariel Gabizon. 2020. Recursive Proof Composition from Accumulation Schemes. Cryptology ePrint Archive, Report 2020/499. https://eprint.iacr.org/2020/499

[6] Dan Boneh, Manu Drijvers, and Gregory Neven. 2018. Compact Multi-Signatures for Smaller Blockchains. In *Advances in Cryptology — ASIACRYPT 2018 (LNCS, Vol. 11273)*. Springer, 435–464. doi:10.1007/978-3-030-03329-3_15

[7] Sean Bowe, Jack Grigg, and Daira Hopwood. 2019. Halo: Recursive Proof Composition without a Trusted Setup. Cryptology ePrint Archive, Report 2019/1021. https://eprint.iacr.org/2019/1021

[8] Conrad Burchert, Christian Decker, and Roger Wattenhofer. 2018. Scalable Funding of Bitcoin Micropayment Channel Networks. In *Royal Society Open Science*, Vol. 5. doi:10.1098/rsos.180089

[9] Jan Camenisch and Anna Lysyanskaya. 2001. Efficient Non-transferable Anonymous Multi-show Credential System with Optional Anonymity Revocation. In *Advances in Cryptology — EUROCRYPT 2001 (LNCS, Vol. 2045)*. Springer, 93–118. doi:10.1007/3-540-44987-6_7

[10] Jan Camenisch and Victor Shoup. 2003. Practical Verifiable Encryption and Decryption of Discrete Logarithms. In *Advances in Cryptology — CRYPTO 2003 (LNCS, Vol. 2729)*. Springer, 126–144. doi:10.1007/978-3-540-45146-4_8

[11] Ran Canetti, Rosario Gennaro, Steven Goldfeder, Nikolaos Makriyannis, and Udi Peled. 2020. UC Non-Interactive, Proactive, Threshold ECDSA with Identifiable Aborts. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 1769–1787. doi:10.1145/3372297.3423367

[12] Ronald Cramer. 1997. *Modular Design of Secure yet Practical Cryptographic Protocols*. Ph.D. Thesis. University of Amsterdam.

[13] Ivan Damgård. 2010. On Σ-Protocols. Lecture notes, Aarhus University. https://www.cs.au.dk/~ivan/Sigma.pdf

[14] Christian Decker, Rusty Russell, and Olaoluwa Osuntokun. 2018. eltoo: A Simple Layer2 Protocol for Bitcoin. White paper. https://blockstream.com/eltoo.pdf

[15] Christian Decker and Roger Wattenhofer. 2015. A Fast and Scalable Payment Network with Bitcoin Duplex Micropayment Channels. In *Stabilization, Safety, and Security of Distributed Systems (SSS) (LNCS, Vol. 9212)*. Springer, 3–18. doi:10.1007/978-3-319-21741-3_1

[16] Stefan Dziembowski, Sebastian Faust, and Kristina Hostáková. 2018. General State Channel Networks. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 949–966. doi:10.1145/3243734.3243856

[17] Christoph Egger, Pedro Moreno-Sanchez, and Matteo Maffei. 2019. Atomic Multi-Channel Updates with Constant Collateral in Bitcoin-Compatible Payment-Channel Networks. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 801–815. doi:10.1145/3319535.3345666

[18] Paul Feldman. 1987. A Practical Scheme for Non-interactive Verifiable Secret Sharing. In *28th Annual Symposium on Foundations of Computer Science (FOCS)*. 427–438. doi:10.1109/SFCS.1987.4

[19] Amos Fiat and Adi Shamir. 1987. How to Prove Yourself: Practical Solutions to Identification and Signature Problems. In *Advances in Cryptology — CRYPTO '86 (LNCS, Vol. 263)*. Springer, 186–194. doi:10.1007/3-540-47721-7_12

[20] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. 2019. PlonK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge. Cryptology ePrint Archive, Report 2019/953. https://eprint.iacr.org/2019/953

[21] Rosario Gennaro, Stanisław Jarecki, Hugo Krawczyk, and Tal Rabin. 2007. Secure Distributed Key Generation for Discrete-Log Based Cryptosystems. *Journal of Cryptology* 20, 1 (2007), 51–83. doi:10.1007/s00145-006-0347-3

[22] Jens Groth. 2016. On the Size of Pairing-Based Non-interactive Arguments. In *Advances in Cryptology — EUROCRYPT 2016 (LNCS, Vol. 9666)*. Springer, 305–326. doi:10.1007/978-3-662-49896-5_11

[23] Lewis Gudgeon, Pedro Moreno-Sanchez, Stefanie Roos, Patrick McCorry, and Arthur Gervais. 2020. SoK: Layer-Two Blockchain Protocols. In *Financial Cryptography and Data Security (FC) (LNCS, Vol. 12059)*. Springer, 201–226. doi:10.1007/978-3-030-51280-4_12

[24] Rami Khalil and Arthur Gervais. 2017. Revive: Rebalancing Off-Blockchain Payment Networks. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 439–453. doi:10.1145/3133956.3134033

[25] Chelsea Komlo and Ian Goldberg. 2021. FROST: Flexible Round-Optimized Schnorr Threshold Signatures. In *Selected Areas in Cryptography (SAC) (LNCS, Vol. 12804)*. Springer, 34–65. doi:10.1007/978-3-030-81652-0_2

[26] Yehuda Lindell. 2017. Fast Secure Two-Party ECDSA Signing. In *Advances in Cryptology — CRYPTO 2017 (LNCS, Vol. 10402)*. Springer, 613–644. doi:10.1007/978-3-319-63715-0_21

[27] Giulio Malavolta, Pedro Moreno-Sanchez, Aniket Kate, Matteo Maffei, and Srivatsan Ravi. 2017. Concurrency and Privacy with Payment-Channel Networks. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 455–471. doi:10.1145/3133956.3134096

[28] Matter Labs. 2023. zkSync: Scaling Freedom for All. Project documentation. https://zksync.io/

[29] Andrew Miller, Iddo Bentov, Surya Bakshi, Ranjit Kumaresan, and Patrick McCorry. 2019. Sprites and State Channels: Payment Networks that Go Faster than Lightning. In *Financial Cryptography and Data Security (FC) (LNCS, Vol. 11598)*. Springer, 508–526. doi:10.1007/978-3-030-32101-7_30

[30] Satoshi Nakamoto. 2008. Bitcoin: A Peer-to-Peer Electronic Cash System. White paper. https://bitcoin.org/bitcoin.pdf

[31] Offchain Labs. 2021. Arbitrum: Scalable, Private Smart Contracts. Project documentation. https://developer.arbitrum.io/

[32] Pascal Paillier. 1999. Public-Key Cryptosystems Based on Composite Degree Residuosity Classes. 1592 (1999), 223–238. doi:10.1007/3-540-48910-X_16

[33] Torben Pryds Pedersen. 1991. Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing. In *Advances in Cryptology — CRYPTO '91 (LNCS, Vol. 576)*. Springer, 129–140. doi:10.1007/3-540-46766-1_9

[34] David Pointcheval and Jacques Stern. 1996. Security Proofs for Signature Schemes. In *Advances in Cryptology — EUROCRYPT '96 (LNCS, Vol. 1070)*. Springer, 387–398. doi:10.1007/3-540-68339-9_33

[35] Joseph Poon and Thaddeus Dryja. 2016. The Bitcoin Lightning Network: Scalable Off-Chain Instant Payments. White paper. https://lightning.network/lightning-network-paper.pdf

[36] RISC Zero. 2024. RISC Zero: General-Purpose Verifiable Computing. Project documentation. https://www.risczero.com/

[37] Tim Ruffing, Viktoria Ronge, Elliott Jin, Jonas Schneider-Bensch, and Dominique Schröder. 2022. ROAST: Robust Asynchronous Schnorr Threshold Signatures. Cryptology ePrint Archive, Report 2022/550. https://eprint.iacr.org/2022/550

[38] Claus-Peter Schnorr. 1991. Efficient Signature Generation by Smart Cards. *Journal of Cryptology* 4, 3 (1991), 161–174. doi:10.1007/BF00196725

[39] Adi Shamir. 1979. How to Share a Secret. *Commun. ACM* 22, 11 (1979), 612–613. doi:10.1145/359168.359176

[40] Victor Shoup. 2024. Simple VESS (Verifiable Encrypted Secret Sharing). Cryptology ePrint Archive, Report 2024/253. https://eprint.iacr.org/2024/253

[41] Jeremy Spilman. 2013. Anti DoS for tx replacement. Bitcoin-development mailing list. https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2013-April/002433.html

[42] Succinct Labs. 2024. SP1: A zkVM for Developers. Project documentation. https://docs.succinct.xyz/

[43] Pieter Wuille, Jonas Nick, and Tim Ruffing. 2020. BIP 340: Schnorr Signatures for secp256k1. Bitcoin Improvement Proposals. https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki

[44] Pieter Wuille, Jonas Nick, and Anthony Towns. 2020. BIP 341: Taproot: SegWit Version 1 Spending Rules. Bitcoin Improvement Proposals. https://github.com/bitcoin/bips/blob/master/bip-0341.mediawiki